

Sparse retrieval represent data using Sparse vectors

- exact keyword modeling (e.g. TF-IDF BM25)
- Sparse embedding (e.g. SPLADE)

TF-IDF (Term-frequency - Inverse document frequency)

TF: Count of a word w_i in a document D_j → intuition: if a word appears frequently in the documentation. This documentation is more likely to be about this word.

$f(w_i, D_j)$

IDF = $\log\left(\frac{N}{DF}\right)$

→ total number of documents

→ the number of document count this word.

→ intuition: if a word appears in all (most) documents, it is probably not very important. e.g. (the, a, an)

$IDF(w_i)$

note this is a function of w_i

How TF-IDF works?

1. Let $w_1, w_2, \dots, w_q$ be the word in the Query.
2. Compute $f(w_i, D_j)$ for all $i = 1, \dots, q, j = 1, \dots, n$
3. Compute $IDF(w_i)$ for all $i = 1, \dots, q$
4. For a given document D_j , the score of TF-IDF for this query is

$$\sum_{i=1}^q f(w_i, D_j) IDF(w_i)$$

note that 1. in TF-IDF, the term frequency in Query is not used (always set to 1)

2. the score of TF-IDF scales linearly with the word frequency in document. If word i appears 10 times more, the score for this word is 10 times longer. However, this bias the retrieval results towards long document. Also, if a word already appears many times, the importance of this word should somehow **saturated** rather than growing infinitely.

BM25 (Best matching 25)

→ improve on TF-IDF

For a given document D_j , the BM25 Score w.r.t the query is

$$\sum_{i=1}^L \text{IDF}(w_i) \frac{f(w_i, D_j) (K_1 + 1)}{f(w_i, D_j) + K_1 \left(1 - b + b \frac{d_j}{\sum_k d_k} \right)}$$

→ normally between 1-2

document length of D_j

e.g. 0.75

average document length

$$= \sum_{i=1}^L \text{IDF}(w_i) \frac{(K_1 + 1)}{1 + K_1 (1 - b + b \frac{d_j}{\sum_k d_k})} \frac{f(w_i, D_j)}{f(w_i, D_j)}$$

penalize if d_j is large

Score w.r.t

SPLADE (Sparse Lexical AND DEVELOPMENT/EXPANSION)

(a combination of Sparse & Dense retrieval)

why? (Vector is Sparse but encode Semantic meaning)

How SPLADE encode query and documents:

Example:

Query: Can you recommend me a cheap laptop? I will mainly use for video games.



fine-tuned BERT

(or other encoder)



get contextual embedding for each word
pay attention to the whole sequence including itself
and then use the embedding to get logits for all words
e.g. if we mask the word "cheap"

we may get logits

- cheap 7.3
- affordable 5.4
- reasonable 2.1

⋮

most words are ≤ 0

If $l_{i,j}$ is the row logit for word j at query position i , the final weight $\tilde{w}_j$ is

$$\tilde{w}_j = \max_{i \in \text{query}} \log(1 + \text{ReLU}(l_{i,j}))$$

→ this filter out where logits are small

then the embedding vector w for query is very sparse

We do the same for each document and get sparse embedding for each document as well.

Then retrieval is just dot product between embeddings of query and documents.

→ this is quick because the embeddings are sparse.

How is Bert fine tuned for SPLADE?

$$\mathcal{L} = \mathcal{L}_{\text{contrastive loss}} + \lambda_q \mathcal{L}_{\text{flaps}}(q) + \lambda_d \mathcal{L}_{\text{flaps}}(d)$$

(across batch)

so if a word is non-zero for one sequence in the batch, penalized loss it penalizes words estimate for many sequences in the batch.

$\tilde{w}_j = \frac{1}{B} \sum_{i=1}^B l_{i,j}$

→ $\mathcal{L}_{\text{flaps}} = \sum_{j \in V} (\tilde{w}_j)^2$

↓ encourage sparsity for query embedding

↓ sparsity for document embedding

batch size B

match

$q_1 \leftrightarrow d_1$

$q_2 \leftrightarrow d_2$

$\vdots$

$q_B \leftrightarrow d_B$

Contrastive loss

$= - \sum_i \log \left(\frac{e^{s(q_i, d_i)}}{\sum_j e^{s(q_i, d_j)}} \right)$

dot product (or cosine similarity) of the embedding of q_i and d_j

→ unlike clip, this loss is not symmetric (scale across documents only)

because we care only about given one query, which document matches the most.

In clip, we care about both direction